



Sistema de Seguridad Universitaria

Manual Técnico de la Plataforma

Arquitectura:	Microservicios con Node.js y MongoDB
Frontend:	React.js con Tailwind CSS Producción
Dominio:	con Nginx + SSL Documentación
Clasificación:	técnica interna
Año:	2026
Aurota:	Gabriela Cordoba Gonzalez

1	Descripción General del Sistema	5
1.1	Propósito	5
1.2	Alcance	5
1.3	Actores del Sistema	6
1.4	Tecnologías Utilizadas	6
2	Arquitectura del Sistema	7
2.1	Visión General	7
2.2	Descripción de Microservicios	8
2.2.1	Auth Service — Puerto 3004	8
2.2.2	Accesos Service — Puerto 3001	8
2.2.3	Incidentes Service — Puerto 3002	8
2.2.4	Objetos Service — Puerto 3003	8
2.2.5	Notificaciones Service — Puerto 3005	8
2.3	Flujo de Autenticación JWT	9
3	Estructura del Proyecto	10
3.1	Organización del Repositorio	10
3.2	Estructura Interna de Cada Microservicio	11
4	Modelos de Base de Datos	12
4.1	Auth Service — Colección usuarios	12
4.2	Accesos Service — Colección registros	12
4.3	Incidentes Service — Colección incidentes	13
4.4	Objetos Service — Colección objetos	14
4.5	Notificaciones Service — Colección notificaciones	14
4.6	Auditoría — Colección auditoria (Auth Service)	15
5	Referencia de la API REST	16
5.1	Convenciones Generales	16
5.2	Auth Service — /api/v1/auth	17
5.3	Accesos Service — /api/v1/accesos	18
5.4	Incidentes Service — /api/v1/incidentes	19
5.5	Objetos Service — /api/v1/objetos	20

5.6	Notificaciones Service — /api/v1/notificaciones	20
6	Middleware de Seguridad	21
6.1	Verificación de Token JWT	21
6.2	Control de Roles	21
7	Flujos Principales del Sistema	23
7.1	Flujo: Registro de Acceso Institucional	23
7.2	Flujo: Reporte de Incidente	23
8	Instalación y Configuración Local	25
8.1	Requisitos Previos	25
8.2	Clonar el Repositorio	25
8.3	Variables de Entorno	26
8.3.1	auth-service/.env	26
8.3.2	accesos-service/.env	26
8.3.3	incidentes-service/.env	26
8.3.4	objetos-service/.env	26
8.3.5	notif-service/.env	27
8.3.6	frontend/.env	27
8.4	Levantar en Modo Desarrollo	27
9	Despliegue en Producción con Dominio y SSL	28
9.1	Resumen de la Infraestructura	28
9.2	Instalar Dependencias en el Servidor	28
9.3	Configuración de PM2	29
9.4	Configuración de Nginx como Reverse Proxy	30
9.5	Obtener Certificado SSL con Let's Encrypt	32
9.6	Variables de Entorno en Producción	32
9.7	Despliegue y Actualización del Código	32
10	Pruebas y Verificación	34
10.1	Pruebas con Postman	34
10.2	Prueba de Conectividad en Producción	34
10.3	Verificación del Certificado SSL	35
11	Mantenimiento y Operación	36
11.1	Monitoreo con PM2	36
11.2	Rotación y Respaldo	36
11.3	Actualizar Certificado SSL Manualmente	36
11.4	Tabla de Puertos del Sistema	37
12	Glosario Técnico	38

2.1	Diagrama de arquitectura de microservicios con MongoDB	7
2.2	Flujo de autenticación y verificación de token JWT	9
7.1	Diagrama de secuencia: registro de acceso institucional	23

Descripción General del Sistema

Propósito

El Sistema de Seguridad Universitaria es una plataforma web desarrollada para centralizar y digitalizar el control de accesos al campus, el reporte y seguimiento de incidentes de seguridad, y la gestión de objetos perdidos o encontrados dentro de la institución.

La plataforma está concebida como un sistema de información operativo que apoya al personal de seguridad (celadores) y a la administración universitaria en la toma de decisiones, al tiempo que proporciona a estudiantes, docentes y personal administrativo canales directos para comunicar situaciones de riesgo o reportar pertenencias extraviadas.

Alcance

El sistema cubre los siguientes módulos funcionales:

- **Control de Accesos:** registro digital de entrada y salida de personas (usuarios institucionales y visitantes externos) con historial completo y estadísticas diarias.
- **Reporte de Incidentes:** formulario de creación de reportes con flujo de estados (Reportado → En revisión → Resuelto) y notificaciones automáticas.
- **Gestión de Objetos Perdidos:** tablán comunitario de publicaciones con comentarios, búsqueda por filtros y marcado como encontrado.
- **Notificaciones:** alertas en tiempo real dirigidas al personal de seguridad y administración.
- **Auditoría:** registro inmutable de todas las acciones realizadas en el sistema para trazabilidad.

- **Administración:** gestión de usuarios, roles y configuración de tipos de incidente.

Actores del Sistema

Rol	Descripción
Usuario	Estudiante, docente o personal administrativo con cuenta en el sistema. Puede reportar incidentes y objetos perdidos.
Visitante	Persona externa sin cuenta. Solo puede ser registrada por el celador.
Celador	Operativo de seguridad. Registra accesos y gestiona reportes.
Administrador	Acceso total: gestiona usuarios, auditoría y configuración del sistema.

Tecnologías Utilizadas

Capa	Tecnología	Descripción
Frontend	React.js + Tailwind CSS	Interfaz de usuario reactiva y responsiva
Backend	Node.js + Express.js	Microservicios REST independientes
Autenticación	JWT (JSON Web Tokens)	Tokens firmados con expiración controlada
Base de datos	MongoDB + Mongoose ODM	Una base de datos por microservicio
Proxy	Nginx	Reverse proxy y terminación SSL
Certificado	Let's Encrypt / Certbot	HTTPS en dominio de producción
Procesos	PM2	Gestor de procesos Node.js en producción
Control de versiones	Git + GitHub	Repositorio y CI del proyecto

CAPÍTULO 2 Arquitectura del Sistema

Visión General

La plataforma implementa una **arquitectura de microservicios**: cada módulo funcional corre como un proceso Node.js independiente, expone su propia API REST y posee su propia base de datos MongoDB. El frontend React.js se comunica únicamente con un **API Gateway** centralizado que enruta las peticiones al servicio correspondiente y valida el token JWT antes de dejarlas pasar.

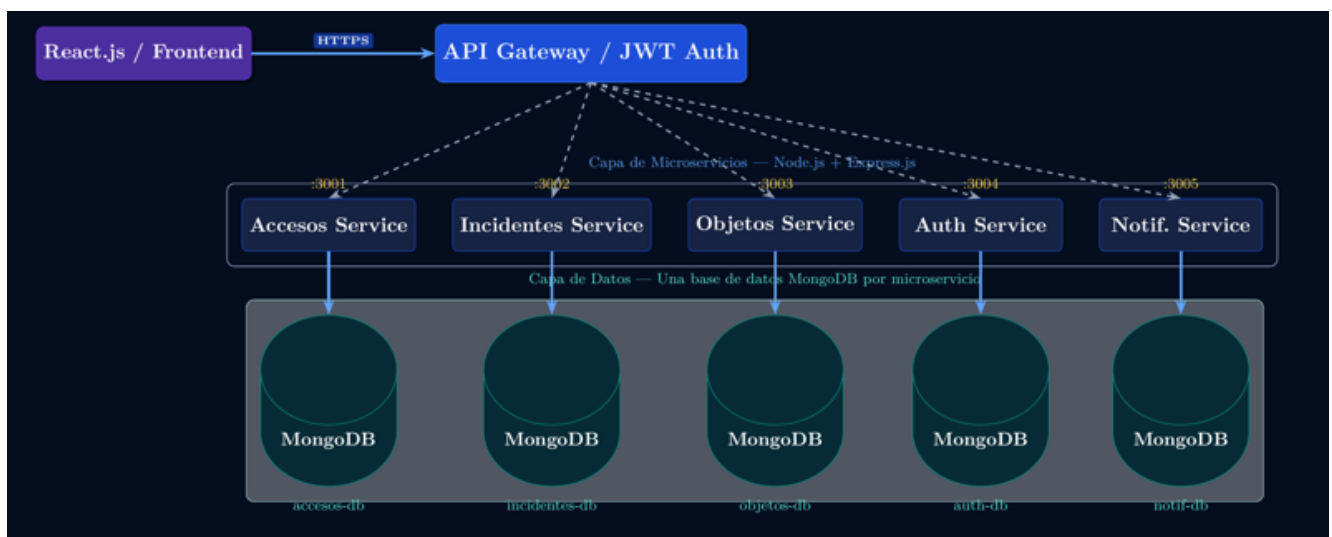


Figura 2.1: Diagrama de arquitectura de microservicios con MongoDB

Nota

Cada microservicio es completamente autónomo: tiene su propio proceso Node.js, su propia base de datos MongoDB y sus propias variables de entorno. Un fallo en un servicio no afecta a los demás.

Descripción de Microservicios

Auth Service — Puerto 3004

Responsable de toda la lógica de autenticación: creación de usuarios, login, generación de tokens JWT y verificación de roles. El resto de microservicios llaman a este servicio para validar tokens entrantes.

Accesos Service — Puerto 3001

Registra y consulta los movimientos de entrada y salida de personas en el campus. Distingue entre usuarios institucionales (validados contra auth-db) y visitantes externos (datos propios sin cuenta).

Incidentes Service — Puerto 3002

Gestiona el ciclo de vida completo de los reportes de seguridad: creación, cambio de estado, asignación de observaciones y resolución. Emite notificaciones al servicio de notificaciones al cambiar estado.

Objetos Service — Puerto 3003

Maneja las publicaciones del tablón de objetos perdidos y encontrados: creación de reportes con imagen opcional, comentarios anidados y marcado como devuelto.

Notificaciones Service — Puerto 3005

Recibe eventos de los demás servicios y los convierte en notificaciones persistentes para los usuarios. Expone endpoints de consulta y marcado como leído.

Flujo de Autenticación JWT

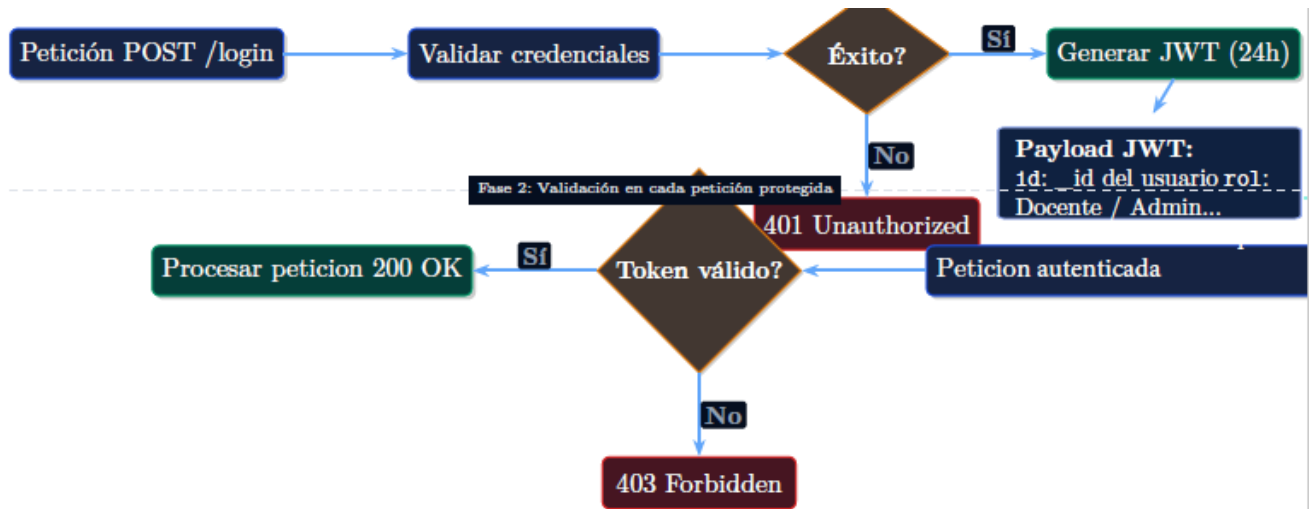


Figura 2.2: Flujo de autenticación y verificación de token JWT

CAPÍTULO 3 Estructura del Proyecto

Organización del Repositorio

```
1 seguridad-universitaria/
2 |-- frontend/                # Aplicacion React.js
3 |   |-- public/
4 |   |-- src/
5 |       |-- components/      # Componentes reutilizables
6 |       |-- pages/           # Vistas por modulo
7 |       |-- auth/
8 |       |-- accesos/
9 |       |-- incidentes/
10 |      |-- objetos/
11 |      |-- admin/
12 |      |-- celador/
13 |      |-- context/          # React Context API (AuthContext)
14 |      |-- services/         # Llamadas Axios a cada microservicio
15 |      |-- hooks/            # Custom hooks
16 |      |-- utils/
17 |      |-- .env
18 |      |-- package.json
19 |
20 |-- services/
21 |   |-- auth-service/        # Puerto 3004
22 |   |-- accesos-service/     # Puerto 3001
23 |   |-- incidentes-service/  # Puerto 3002
24 |   |-- objetos-service/     # Puerto 3003
25 |   |-- notif-service/       # Puerto 3005
26 |
27 |-- nginx/
28 |   |-- nginx.conf           # Configuracion del reverse proxy
29 |   |-- ssl/                  # Certificados Let's Encrypt
```

Listing 3.1: Árbol de directorios del repositorio

Estructura Interna de Cada Microservicio

Todos los microservicios siguen la misma convención de estructura:

```
1 auth-service/  
2 |-- src/  
3 |   |-- config/  
4 |   |   '-- db.js           # Conexion a MongoDB  
5 |   |-- controllers/       # Logica de negocio por recurso  
6 |   |-- middlewares/  
7 |   |   |-- authMiddleware.js # Verificacion JWT  
8 |   |   '-- roleMiddleware.js # Control por rol  
9 |   |-- models/           # Esquemas Mongoose  
10 |  |-- routes/            # Definicion de rutas Express  
11 |  '-- utils/             # Helpers (bcrypt, jwt, etc.)  
12 |-- .env                  # Variables de entorno  
13 |-- server.js              # Punto de entrada  
14 '-- package.json
```

Listing 3.2: Estructura interna de un microservicio

CAPÍTULO 4 Modelos de Base de Datos

Auth Service — Colección usuarios

```
1 const usuarioSchema = new mongoose.Schema({
2   nombres:      { type: String, required: true, trim: true },
3   apellidos:    { type: String, required: true, trim: true },
4   numeroDocumento: { type: String, required: true, unique: true },
5   tipoDocumento: {
6     type: String,
7     enum: ['CC', 'TI', 'CE', 'Pasaporte'],
8     default: 'CC'
9   },
10  correo:        { type: String, required: true, unique: true,
11                  lowercase: true },
12  telefono:      { type: String },
13  password:      { type: String, required: true }, // bcrypt hash
14  rol:           {
15    type: String,
16    enum: ['Estudiante', 'Docente', 'Personal Administrativo',
17          'Celador', 'Administrador'],
18    required: true
19  },
20  estado:        { type: String, enum: ['Activo', 'Bloqueado'],
21                  default: 'Activo' },
22 }, { timestamps: true });
```

Listing 4.1: Esquema Mongoose del modelo Usuario

Nota

La contraseña se almacena siempre como hash **bcrypt** con factor de coste 10. Nunca se persiste en texto plano.

Accesos Service — Colección registros

```

1 const registroAccesoSchema = new mongoose.Schema({
2   tipo: { type: String, enum: ['Institucional', 'Visitante'],
3         required: true },
4   movimiento: { type: String, enum: ['Entrada', 'Salida'],
5               required: true },
6   // Campos para usuario institucional
7   usuarioId: { type: String }, // _id del Auth Service
8   numeroDocumento: { type: String },
9   nombreCompleto: { type: String },
10  // Campos para visitante externo
11  visitante: {
12    documento: { type: String },
13    nombres: { type: String },
14    apellidos: { type: String },
15    motivoVisita: { type: String },
16    personaVisitar: { type: String },
17  },
18  observaciones: { type: String },
19  registradoPor: { type: String, required: true }, // _id del celador
20  fecha: { type: Date, default: Date.now },
21 }, { timestamps: true });

```

Listing 4.2: Esquema Mongoose del modelo RegistroAcceso

Incidentes Service — Colección incidentes

```

1 const incidenteSchema = new mongoose.Schema({
2   tipo: { type: String, required: true },
3   severidad: {
4     type: String,
5     enum: ['Baja', 'Media', 'Alta', 'Critica'],
6     required: true
7   },
8   ubicacion: { type: String, required: true },
9   fechaIncidente: { type: Date, required: true },
10  descripcion: { type: String, required: true, maxlength: 1000 },
11  estado: {
12    type: String,
13    enum: ['Reportado', 'En Revision', 'Resuelto'],
14    default: 'Reportado'
15  },
16  reportadoPor: {
17    usuarioId: String,
18    nombre: String,
19    documento: String,
20    rol: String,
21  },
22  gestionadoPor: {
23    usuarioId: String,
24    nombre: String,
25  },
26  observacionesGestion: { type: String },
27  fechaResolucion: { type: Date },

```

```
28 }, { timestamps: true });
```

Listing 4.3: Esquema Mongoose del modelo Incidente

Objetos Service — Colección objetos

```
1 const objetoSchema = new mongoose.Schema({
2   tipoReporte: { type: String, enum: ['Perdido', 'Encontrado'],
3                 required: true },
4   nombreObjeto: { type: String, required: true },
5   descripcion: { type: String, required: true, maxlength: 600 },
6   ubicacion: { type: String, required: true },
7   fechaEvento: { type: Date, required: true },
8   imagen: { type: String, // URL o base64
9   estado: {
10     type: String,
11     enum: ['Activo', 'Devuelto'],
12     default: 'Activo'
13   },
14   publicadoPor: {
15     usuarioId: String,
16     nombre: String,
17     rol: String,
18   },
19   comentarios: [{
20     usuarioId: String,
21     nombre: String,
22     rol: String,
23     texto: { type: String, required: true },
24     fecha: { type: Date, default: Date.now },
25   }],
26 }, { timestamps: true });
```

Listing 4.4: Esquema Mongoose del modelo ObjetoPerdido

Notificaciones Service — Colección notificaciones

```
1 const notificacionSchema = new mongoose.Schema({
2   tipo: {
3     type: String,
4     enum: ['NuevoIncidente', 'ObjetoPerdido', 'CambioEstado',
5           'NuevoComentario'],
6     required: true
7   },
8   titulo: { type: String, required: true },
9   mensaje: { type: String, required: true },
10  leido: { type: Boolean, default: false },
11  usuarioId: { type: String, // null = notif. global
12  referenciaId: { type: String, // _id del objeto/incidente
13 }, { timestamps: true });
```

Listing 4.5: Esquema Mongoose del modelo Notificacion

Auditoría — Colección **auditoria** (Auth Service)

```
1 const auditoriaSchema = new mongoose.Schema({
2   usuarioId:    { type: String, required: true },
3   nombreUsuario:{ type: String },
4   accion:       { type: String, required: true },
5   modulo:       {
6     type: String,
7     enum: ['Accesos', 'Incidentes', 'Objetos', 'Usuarios',
8           'Configuracion', 'Auth'],
9     required: true
10  },
11  descripcion:   { type: String },
12  ip:            { type: String },
13  fecha:         { type: Date, default: Date.now },
14 }, { timestamps: false });
```

Listing 4.6: Esquema de registro de auditoría

CAPÍTULO 5 Referencia de la API REST

Convenciones Generales

- Todas las rutas usan el prefijo `/api/v1/`.
- El cuerpo de las peticiones y respuestas es siempre `application/json`.
- Las rutas protegidas requieren el encabezado `Authorization: Bearer <token>`.
- Los errores devuelven siempre `{"message": "descripcion"}`.
- Las fechas se transmiten en formato ISO 8601.

Códigos de respuesta utilizados

Código	Estado	Descripción
200	OK	Operación exitosa
201	Created	Recurso creado exitosamente
400	Bad Request	Parámetros inválidos o faltantes
401	Unauthorized	Token ausente, inválido o expirado
403	Forbidden	Token válido pero rol insuficiente
404	Not Found	Recurso no encontrado
409	Conflict	Conflicto (ej. documento ya registrado)
500	Internal Server Error	Error inesperado del servidor

Auth Service — `/api/v1/auth`

Método	Ruta	Rol	Descripción
POST	/login	PÚBLICO	Autenticar usuario y obtener JWT
POST	/usuarios	ADMIN	Crear nuevo usuario
GET	/usuarios	ADMIN	Listar todos los usuarios
GET	/usuarios/:id	ADMIN	Obtener usuario por ID
PUT	/usuarios/:id	ADMIN	Editar datos del usuario
PATCH	/usuarios/:id/estado	ADMIN	Bloquear o activar usuario
DELETE	/usuarios/:id	ADMIN	Eliminar usuario
GET	/auditoria	ADMIN	Consultar registros de auditoría
GET	/configuracion	ADMIN	Obtener tipos de incidente y roles
POST	/configuracion/tipo	ADMIN	Agregar tipo de incidente
DELETE	/configuracion/tipo/	ADMIN	Eliminar tipo de incidente

Ejemplo: POST /api/v1/auth/login

```

1 {
2   "numeroDocumento": "1078456675",
3   "password": "miPassword123"
4 }
```

Listing 5.1: Body de la petición de login

```

1 {
2   "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
3   "usuario": {
4     "_id": "64a7f2e3b1c4e5d6f7a8b9c0",
5     "nombres": "Andres",
6     "apellidos": "Palacios Palacios",
7     "rol": "Docente",
8     "estado": "Activo"
9   }
10 }
```

Listing 5.2: Respuesta exitosa del login (200 OK)

Accesos Service — /api/v1/accesos

Método	Ruta	Rol	Descripción
POST	/institucional	CELADOR ADMIN	Registrar acceso de usuario
POST	/visitante	CELADOR ADMIN	Registrar acceso de visitante
GET	/	CELADOR ADMIN	Historial completo con filtros
GET	/hoy	CELADOR ADMIN	Registros del día actual
GET	/estadisticas	ADMIN	Estadísticas por rango de fecha

Ejemplo: POST /api/v1/accesos/institucional

```
1 {  
2   "numeroDocumento": "1078458186",  
3   "movimiento": "Entrada",  
4   "observaciones": "Ninguna"  
5 }
```

Listing 5.3: Body para registrar acceso institucional

Incidentes Service — /api/v1/incidentes

Método	Ruta	Rol	Descripción
POST	/	USUARIO CELADOR ADMIN	Crear reporte
GET	/	CELADOR ADMIN	Todos los reportes
GET	/mis-reportes	USUARIO	Reportes del usuario
GET	/:id	USUARIO CELADOR ADMIN	Detalle del reporte
PATCH	/:id/revision	CELADOR ADMIN	Tomar en revisión
PATCH	/:id/resolver	CELADOR ADMIN	Marcar como resuelto
PUT	/:id	ADMIN	Editar reporte

Ejemplo: POST `/api/v1/incidentes/`

```

1 {
2   "tipo": "Accidente",
3   "ubicacion": "Segundo piso, salon 801",
4   "fechaIncidente": "2026-05-25T03:10:00.000Z",
5   "descripcion": "El piso estaba resbaloso y ocurrio una caida."
6 }
```

Listing 5.4: Body para crear un reporte de incidente

Objetos Service — `/api/v1/objetos`

Método	Ruta	Rol	Descripción
POST	/	USUARIO CELADOR ADMIN	Publicar objeto
GET	/	Todos los roles	Listar objetos con filtros
GET	/:id	Todos los roles	Detalle del objeto
PATCH	/:id/encontrado	ADMIN	Marcar como encontrado
POST	/:id/comentarios	Todos los roles	Agregar comentario
DELETE	:id	ADMIN	Eliminar publicación

Notificaciones Service — [/api/v1/notificaciones](#)

Método	Ruta	Rol	Descripción
GET	/	Todos los roles	Mis notificaciones
PATCH	/:id/leer	Todos los roles	Marcar una como leída
PATCH	/leer-todas	Todos los roles	Marcar todas como leídas

CAPÍTULO 6 Middleware de Seguridad

Verificación de Token JWT

```
1 const jwt = require('jsonwebtoken');
2
3 const authMiddleware = (req, res, next) => {
4   const authHeader = req.headers.authorization;
5
6   if (!authHeader || !authHeader.startsWith('Bearer ')) {
7     return res.status(401).json({ message: 'Token requerido' });
8   }
9
10  const token = authHeader.split(' ')[1];
11
12  try {
13    const decoded = jwt.verify(token, process.env.JWT_SECRET);
14    req.usuario = decoded; // { id, rol, nombre, ... }
15    next();
16  } catch (err) {
17    return res.status(401).json({ message: 'Token invalido o expirado' });
18  }
19 };
20
21 module.exports = authMiddleware;
```

Listing 6.1: Middleware de autenticación JWT

Control de Roles

```
1 const roleMiddleware = (...rolesPermitidos) => {
2   return (req, res, next) => {
3     if (!req.usuario) {
4       return res.status(401).json({ message: 'No autenticado' });
5     }
6   }
7 }
```

```
6
7   if (!rolesPermitidos.includes(req.usuario.rol)) {
8     return res.status(403).json({
9       message: 'Acceso denegado. Rol requerido:
10         ${rolesPermitidos.join(' o ')}'
11     });
12   }
13   next();
14 };
15 };
16
17 module.exports = roleMiddleware;
```

Listing 6.2: Middleware de control por rol

Uso combinado en rutas

```
1 const auth = require('../middlewares/authMiddleware');
2 const role = require('../middlewares/roleMiddleware');
3
4 // Solo administradores
5 router.get('/usuarios', auth, role('Administrador'), listarUsuarios);
6
7 // Celadores y administradores
8 router.post('/accesos', auth, role('Celador', 'Administrador'),
9   registrar);
10
11 // Cualquier usuario autenticado
12 router.post('/incidentes', auth, crearIncidente);
```

Listing 6.3: Aplicación de middlewares en una ruta Express

Flujos Principales del Sistema

Flujo: Registro de Acceso Institucional

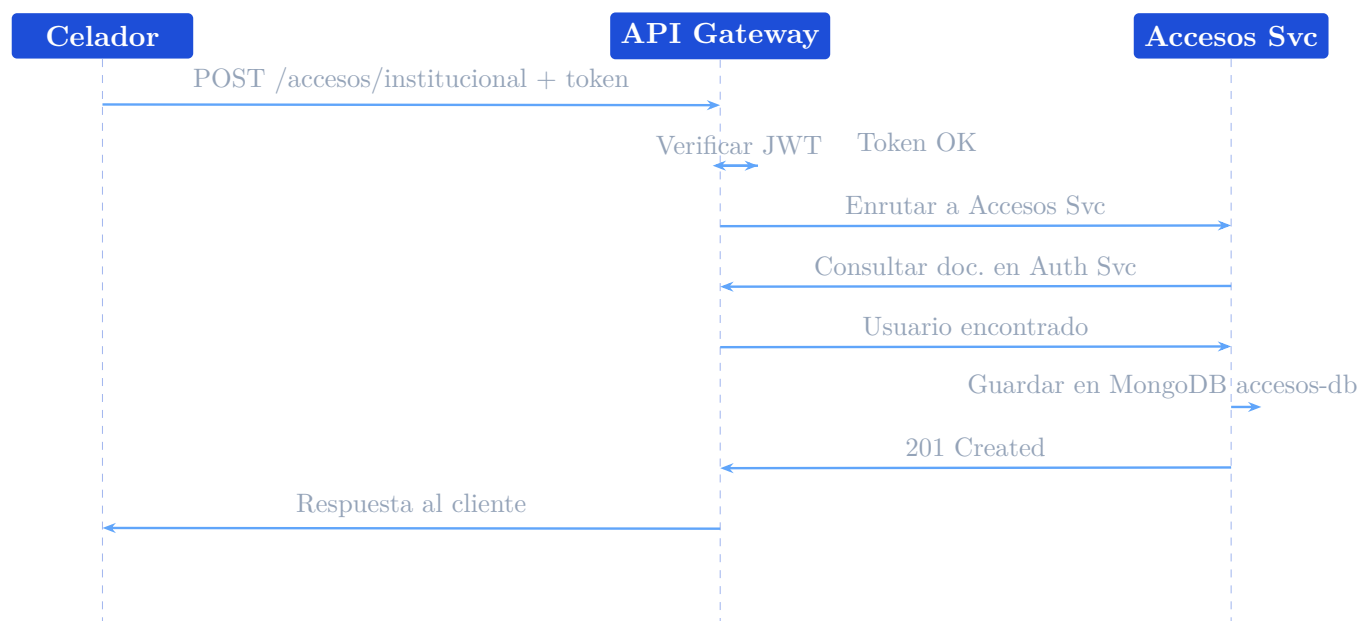


Figura 7.1: Diagrama de secuencia: registro de acceso institucional

Flujo: Reporte de Incidente

1. El usuario autenticado envía `POST /api/v1/incidentes` con los datos del incidente.
2. El API Gateway verifica el token JWT y comprueba que el rol sea *Usuario*, *Celador* o *Administrador*.

3. El Incidentes Service valida los campos requeridos, determina la severidad según el tipo y persiste el documento en `incidentes-db` con estado `Reportado`.
4. El Incidentes Service emite un evento interno al Notificaciones Service con los datos del nuevo reporte.
5. El Notificaciones Service crea notificaciones en `notif-db` dirigidas a todos los usuarios con rol *Celador* y *Administrador*.
6. La respuesta regresa al cliente con el documento creado y estado `201 Created`.

Nota

El cambio de estado de *Reportado* a *En Revisión* y luego a *Resuelto* sólo lo pueden realizar Celadores y Administradores mediante los endpoints `PATCH /:id/revision` y `PATCH /:id/resolver`.

Instalación y Configuración

Local

Requisitos Previos

Herramienta	Versión mínima	Notas
Node.js	18.x LTS	Gestiona todos los microservicios
npm	9.x	Incluido con Node.js
MongoDB	6.x	Local o MongoDB Atlas
Git	2.x	Para clonar el repositorio

Clonar el Repositorio

```
1 git clone https://github.com/usuario/seguridad-universitaria.git
2 cd seguridad-universitaria
3
4 # Instalar dependencias del frontend
5 cd frontend && npm install && cd ..
6
7 # Instalar dependencias de cada microservicio
8 for svc in auth-service accesos-service incidentes-service \
9     objetos-service notif-service; do
10     cd services/$svc && npm install && cd ../..
11 done
```

Listing 8.1: Clonar e instalar dependencias

Variables de Entorno

Cada microservicio requiere su propio archivo `.env`. A continuación se detallan las variables de cada uno.

auth-service/.env

```
1 PORT=3004
2 MONGODB_URI=mongodb://localhost:27017/auth-db
3 JWT_SECRET=tu_clave_secreta_muy_larga_y_segura
4 JWT_EXPIRES_IN=24h
5 NODE_ENV=development
```

Listing 8.2: Variables de entorno del Auth Service

accesos-service/.env

```
1 PORT=3001
2 MONGODB_URI=mongodb://localhost:27017/accesos-db
3 JWT_SECRET=tu_clave_secreta_muy_larga_y_segura
4 AUTH_SERVICE_URL=http://localhost:3004
5 NODE_ENV=development
```

Listing 8.3: Variables de entorno del Accesos Service

incidentes-service/.env

```
1 PORT=3002
2 MONGODB_URI=mongodb://localhost:27017/incidentes-db
3 JWT_SECRET=tu_clave_secreta_muy_larga_y_segura
4 NOTIF_SERVICE_URL=http://localhost:3005
5 NODE_ENV=development
```

Listing 8.4: Variables de entorno del Incidentes Service

objetos-service/.env

```
1 PORT=3003
2 MONGODB_URI=mongodb://localhost:27017/objetos-db
3 JWT_SECRET=tu_clave_secreta_muy_larga_y_segura
4 NOTIF_SERVICE_URL=http://localhost:3005
5 NODE_ENV=development
```

Listing 8.5: Variables de entorno del Objetos Service

notif-service/.env

```
1 PORT=3005
2 MONGODB_URI=mongodb://localhost:27017/notif-db
3 JWT_SECRET=tu_clave_secreta_muy_larga_y_segura
4 NODE_ENV=development
```

Listing 8.6: Variables de entorno del Notificaciones Service**frontend/.env**

```
1 REACT_APP_AUTH_URL=http://localhost:3004/api/v1/auth
2 REACT_APP_ACCESOS_URL=http://localhost:3001/api/v1/accesos
3 REACT_APP_INCIDENTES_URL=http://localhost:3002/api/v1/incidentes
4 REACT_APP_OBJETOS_URL=http://localhost:3003/api/v1/objetos
5 REACT_APP_NOTIF_URL=http://localhost:3005/api/v1/notificaciones
```

Listing 8.7: Variables de entorno del Frontend React.js**Levantar en Modo Desarrollo**

```
1 # Terminal 1 - Auth Service
2 cd services/auth-service && npm run dev
3
4 # Terminal 2 - Accesos Service
5 cd services/accesos-service && npm run dev
6
7 # Terminal 3 - Incidentes Service
8 cd services/incidentes-service && npm run dev
9
10 # Terminal 4 - Objetos Service
11 cd services/objetos-service && npm run dev
12
13 # Terminal 5 - Notificaciones Service
14 cd services/notif-service && npm run dev
15
16 # Terminal 6 - Frontend React
17 cd frontend && npm start
```

Listing 8.8: Inicio de todos los servicios en desarrollo**Consejo**

Para simplificar el arranque en desarrollo, se puede usar `concurrently` o `pm2` en modo desarrollo ejecutando todos los servicios desde la raíz del proyecto con un único script `npm run dev:all`.

CAPÍTULO 9 Despliegue en Producción con Dominio y SSL

Resumen de la Infraestructura

Componente	Detalle
Servidor VPS	Ubuntu 22.04 LTS
Dominio	seguridaduniversitaria.com
Certificado SSL	Let's Encrypt (renovación automática con Certbot)
Reverse Proxy	Nginx 1.24
Gestor de procesos	PM2
Base de datos	MongoDB Atlas (cluster compartido M0)

Instalar Dependencias en el Servidor

```
1 # Actualizar el sistema
2 sudo apt update && sudo apt upgrade -y
3
4 # Instalar Node.js 18 LTS via NodeSource
5 curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
6 sudo apt install -y nodejs
7
8 # Instalar PM2 globalmente
9 sudo npm install -g pm2
10
11 # Instalar Nginx
12 sudo apt install -y nginx
13
14 # Instalar Certbot para Let's Encrypt
```

```
15 sudo apt install -y certbot python3-certbot-nginx
```

Listing 9.1: Instalación del entorno en Ubuntu 22.04

Configuración de PM2

PM2 mantiene todos los microservicios corriendo en segundo plano y los reinicia automáticamente si fallan o si el servidor se reinicia.

```
1 module.exports = {
2   apps: [
3     {
4       name: 'auth-service',
5       script: './services/auth-service/server.js',
6       env_production: {
7         NODE_ENV: 'production',
8         PORT: 3004,
9       },
10      instances: 1,
11      autorestart: true,
12      watch: false,
13    },
14    {
15      name: 'accesos-service',
16      script: './services/accesos-service/server.js',
17      env_production: { NODE_ENV: 'production', PORT: 3001 },
18      instances: 1, autorestart: true, watch: false,
19    },
20    {
21      name: 'incidentes-service',
22      script: './services/incidentes-service/server.js',
23      env_production: { NODE_ENV: 'production', PORT: 3002 },
24      instances: 1, autorestart: true, watch: false,
25    },
26    {
27      name: 'objetos-service',
28      script: './services/objetos-service/server.js',
29      env_production: { NODE_ENV: 'production', PORT: 3003 },
30      instances: 1, autorestart: true, watch: false,
31    },
32    {
33      name: 'notif-service',
34      script: './services/notif-service/server.js',
35      env_production: { NODE_ENV: 'production', PORT: 3005 },
36      instances: 1, autorestart: true, watch: false,
37    },
38  ],
39 };
```

Listing 9.2: Archivo ecosystem.config.js de PM2

```
1 # Iniciar todos los servicios en modo produccion
2 pm2 start ecosystem.config.js --env production
```

```
3
4 # Guardar configuracion para reinicio automatico del sistema
5 pm2 save
6 pm2 startup
7
8 # Ver estado de todos los procesos
9 pm2 status
10
11 # Ver logs en tiempo real
12 pm2 logs
13
14 # Reiniciar un servicio especifico
15 pm2 restart incidentes-service
```

Listing 9.3: Comandos PM2 principales

Configuración de Nginx como Reverse Proxy

Nginx actúa como puerta de entrada única: recibe todas las peticiones HTTP/HTTPS en el dominio y las enruta al microservicio correspondiente según el prefijo de la URL.

```
1 server {
2     listen 80;
3     server_name seguridaduniversitaria.com
4         www.seguridaduniversitaria.com;
5
6     # Redirigir todo HTTP a HTTPS
7     return 301 https://$host$request_uri;
8 }
9
10 server {
11     listen 443 ssl;
12     server_name seguridaduniversitaria.com
13         www.seguridaduniversitaria.com;
14
15     # Certificados Let's Encrypt (gestionados por Certbot)
16     ssl_certificate
17         /etc/letsencrypt/live/seguridaduniversitaria.com/fullchain.pem;
18     ssl_certificate_key
19         /etc/letsencrypt/live/seguridaduniversitaria.com/privkey.pem;
20     include /etc/letsencrypt/options-ssl-nginx.conf;
21     ssl_dhparam /etc/letsencrypt/ssl-dhparams.pem;
22
23     # Cabeceras de seguridad
24     add_header Strict-Transport-Security "max-age=31536000" always;
25     add_header X-Frame-Options DENY;
26     add_header X-Content-Type-Options nosniff;
27
28     # Auth Service
29     location /api/v1/auth {
30         proxy_pass http://localhost:3004;
31         proxy_http_version 1.1;
32         proxy_set_header Host $host;
33         proxy_set_header X-Real-IP $remote_addr;
```

```
30     proxy_set_header    X-Forwarded-For $proxy_add_x_forwarded_for;
31     proxy_set_header    X-Forwarded-Proto $scheme;
32 }
33
34 # Accesos Service
35 location /api/v1/accesos {
36     proxy_pass http://localhost:3001;
37     proxy_set_header Host $host;
38     proxy_set_header X-Real-IP $remote_addr;
39 }
40
41 # Incidentes Service
42 location /api/v1/incidentes {
43     proxy_pass http://localhost:3002;
44     proxy_set_header Host $host;
45     proxy_set_header X-Real-IP $remote_addr;
46 }
47
48 # Objetos Service
49 location /api/v1/objetos {
50     proxy_pass http://localhost:3003;
51     proxy_set_header Host $host;
52     proxy_set_header X-Real-IP $remote_addr;
53 }
54
55 # Notificaciones Service
56 location /api/v1/notificaciones {
57     proxy_pass http://localhost:3005;
58     proxy_set_header Host $host;
59     proxy_set_header X-Real-IP $remote_addr;
60 }
61
62 # Frontend React.js (build estatico)
63 location / {
64     root /var/www/seguridad-universitaria/frontend/build;
65     index index.html;
66     try_files $uri $uri/ /index.html;
67 }
68 }
```

Listing 9.4: Archivo `/etc/nginx/sites-available/seguridad`

```
1 # Crear enlace simbolico para activar el sitio
2 sudo ln -s /etc/nginx/sites-available/seguridad \
3     /etc/nginx/sites-enabled/seguridad
4
5 # Verificar que la configuracion es valida
6 sudo nginx -t
7
8 # Recargar Nginx sin interrumpir conexiones
9 sudo systemctl reload nginx
```

Listing 9.5: Activar la configuración de Nginx

Obtener Certificado SSL con Let's Encrypt

```
1 # Emitir el certificado (Certbot modifica nginx.conf automaticamente)
2 sudo certbot --nginx -d seguridaduniversitaria.com \
3     -d www.seguridaduniversitaria.com
4
5 # Verificar renovacion automatica (se ejecuta dos veces al dia via cron)
6 sudo certbot renew --dry-run
```

Listing 9.6: Emisión del certificado SSL con Certbot

Nota

Let's Encrypt emite certificados válidos por **90 días**. Certbot instala automáticamente un cron job que renueva el certificado cuando quedan menos de 30 días de validez. No es necesario hacerlo manualmente.

Variables de Entorno en Producción

En producción, la variable `MONGODB_URI` de cada servicio debe apuntar al cluster de **MongoDB Atlas** y todas las `*_URL` entre servicios deben usar `localhost` (los servicios corren en el mismo servidor).

```
1 PORT=3002
2 MONGODB_URI=mongodb+srv://usuario:password@cluster0.xxxxx.mongodb.net/inc
3 JWT_SECRET=clave_muy_larga_y_aleatoria_produccion
4 NOTIF_SERVICE_URL=http://localhost:3005
5 NODE_ENV=production
```

Listing 9.7: Ejemplo .env de producción para incidentes-service

Importante

Nunca incluir los archivos `.env` en el repositorio Git. Agregar `.env` al archivo `.gitignore` de cada servicio. Las variables de producción deben gestionarse manualmente en el servidor o mediante un gestor de secretos.

Despliegue y Actualización del Código

```
1 # 1. Conectarse al servidor
2 ssh usuario@ip-del-servidor
3
4 # 2. Ir al directorio del proyecto
5 cd /var/www/seguridad-universitaria
6
7 # 3. Obtener los ultimos cambios
8 git pull origin main
9
10 # 4. Reinstalar dependencias si hubo cambios en package.json
11 cd services/incidentes-service && npm install --production && cd ../..
```



```
12
13 # 5. Reconstruir el frontend
14 cd frontend && npm run build && cd ..
15
16 # 6. Reiniciar los servicios afectados con PM2
17 pm2 restart incidentes-service
18
19 # 7. Verificar que todo esta corriendo correctamente
20 pm2 status
21 pm2 logs --lines 50
```

Listing 9.8: Proceso de actualización del sistema en producción

Pruebas y Verificación

Pruebas con Postman

Durante el desarrollo se utilizaron colecciones de **Postman** para verificar cada endpoint antes de integrarlo al frontend. La colección completa se encuentra en la raíz del repositorio como `postman_collection.json`.

Variables de entorno Postman recomendadas

Variable	Valor
base_auth	http://localhost:3004/api/v1/auth
base_accesos	http://localhost:3001/api/v1/accesos
base_incidentes	http://localhost:3002/api/v1/incidentes
base_objetos	http://localhost:3003/api/v1/objetos
base_notif	http://localhost:3005/api/v1/notificaciones
token	(se actualiza automáticamente tras el login)

Prueba de Conectividad en Producción

```
1 # Verificar HTTPS y redireccion desde HTTP
2 curl -I http://seguridaduniversitaria.com
3 # Esperado: 301 Moved Permanently -> https://...
4
5 curl -I https://seguridaduniversitaria.com
6 # Esperado: 200 OK (pagina React)
7
8 # Probar un endpoint de la API
9 curl -X POST https://seguridaduniversitaria.com/api/v1/auth/login \
```

```
10 -H "Content-Type: application/json" \  
11 -d '{"numeroDocumento":"123456789","password":"password123"}'  
12 # Esperado: {"token":"eyJ...", "usuario":{"...}}  
13  
14 # Verificar PM2  
15 pm2 status  
16 # Todos los procesos deben estar en estado "online"
```

Listing 10.1: Verificar que todos los servicios responden en producción

Verificación del Certificado SSL

```
1 # Ver fecha de expiracion del certificado  
2 sudo certbot certificates  
3  
4 # Salida esperada:  
5 # Certificate Name: seguridaduniversitaria.com  
6 # Domains: seguridaduniversitaria.com www.seguridaduniversitaria.com  
7 # Expiry Date: 2026-08-15 (VALID: 89 days)  
8 # Certificate Path: /etc/letsencrypt/live/.../fullchain.pem
```

Listing 10.2: Verificar validez del certificado SSL

Mantenimiento y Operación

Monitoreo con PM2

```
1 # Panel de monitoreo interactivo (CPU, RAM, logs)
2 pm2 monit
3
4 # Ver logs de un servicio específico
5 pm2 logs auth-service --lines 100
6
7 # Reiniciar todos los servicios
8 pm2 restart all
9
10 # Ver metricas de uso de recursos
11 pm2 show incidentes-service
```

Listing 11.1: Comandos de monitoreo y mantenimiento PM2

Rotación y Respaldo

```
1 # Descargar un dump de la base de produccion
2 mongodump \
3   --uri="mongodb+srv://usuario:pass@cluster0.xxx.mongodb.net/incidentes-db"
4   --out=/backups/$(date +%Y-%m-%d) /
5
6 # Los backups automaticos se configuran directamente en MongoDB Atlas
7 # Dashboard > Backups > Enable Continuous Backup
```

Listing 11.2: Respaldo manual de MongoDB Atlas via mongodump

Actualizar Certificado SSL Manualmente

```

1 # Forzar renovacion antes de los 30 dias (solo para pruebas)
2 sudo certbot renew --force-renewal
3
4 # Recargar Nginx despues de la renovacion
5 sudo systemctl reload nginx

```

Listing 11.3: Renovación manual del certificado si fuera necesario

Tabla de Puertos del Sistema

Servicio	Puerto	Descripción
Nginx (HTTP)	80	Redirige a HTTPS
Nginx (HTTPS)	443	Entrada pública del sistema
Frontend (build)	--	Servido como estático por Nginx
Auth Service	3004	Autenticación, usuarios, auditoría
Accesos Service	3001	Registro de entradas y salidas
Incidentes Service	3002	Gestión de reportes de seguridad
Objetos Service	3003	Tablón de objetos perdidos
Notif. Service	3005	Notificaciones y alertas
MongoDB (local dev)	27017	Solo entorno de desarrollo

Importante

En producción, **solo los puertos 80 y 443 deben estar expuestos** al exterior. Los puertos 3001–3005 deben estar cerrados en el firewall del servidor (`ufw`) y únicamente accesibles desde `localhost`. Nginx es el único punto de entrada.

```

1 # Permitir solo SSH, HTTP y HTTPS
2 sudo ufw allow OpenSSH
3 sudo ufw allow 'Nginx Full'
4 sudo ufw enable
5
6 # Verificar reglas activas
7 sudo ufw status verbose
8 # Debe mostrar solo 22 (SSH), 80 (HTTP) y 443 (HTTPS) abiertos

```

Listing 11.4: Configuración del firewall UFW en producción

Glosario Técnico

API Gateway	Componente central que recibe todas las peticiones del cliente, verifica la autenticación y las enruta al microservicio correspondiente.
JWT	<i>JSON Web Token</i> . Estándar RFC 7519 para transmitir información de forma segura como objeto JSON firmado criptográficamente.
Microservicio	Componente de software independiente que implementa una única capacidad de negocio, con su propio proceso, base de datos y ciclo de despliegue.
Mongoose	Biblioteca ODM (<i>Object Document Mapper</i>) para Node.js que proporciona modelado de esquemas, validación y consultas sobre MongoDB.
Nginx	Servidor web y proxy inverso de alto rendimiento. En este sistema actúa como terminador SSL y enrutador de tráfico hacia los microservicios.
PM2	Gestor de procesos para aplicaciones Node.js en producción. Garantiza que los servicios se reinicien automáticamente tras un fallo o reinicio del servidor.
Let's Encrypt	Autoridad certificadora sin costo que emite certificados SSL/TLS de 90 días con renovación automática.
Certbot	Herramienta cliente de Let's Encrypt que automatiza la emisión y renovación de certificados SSL e integra con Nginx.
Reverse Proxy	Servidor intermediario que recibe peticiones externas y las reenvía a servidores internos, ocultando la topología del backend.
MongoDB Atlas	Servicio gestionado de base de datos MongoDB en la nube con backups automáticos, monitoreo y alta disponibilidad.
bcrypt	Algoritmo de hash diseñado para almacenar contraseñas de forma segura con factor de coste configurable.

ODM	<i>Object Document Mapper</i> . Patrón de abstracción que mapea documentos de MongoDB a objetos JavaScript.
UFW	<i>Uncomplicated Firewall</i> . Interfaz simplificada para <code>iptables</code> en sistemas Ubuntu/Debian.